Capitolo 11

Evoluzione, manutenzione, metriche e difetti

11.1 Il cambiamento dopo la consegna

Un prodotto software consegnato non è necessariamente un prodotto concluso. Il software è flessibile per sua natura e, quando cambiano i requisiti o l'ambiente di uso — per esempio business o hardware — deve cambiare per continuare a essere utile. La **manutenzione** è il processo con cui si modifica un prodotto software dopo la consegna al cliente.

Il termine **evoluzione** è spesso usato come sinonimo di manutenzione; può anche indicare un singolo intervento di cambiamento all'interno di un ciclo che comprende il rilascio di patch e l'eventuale dismissione del sistema.

Una **metrica** permette di osservare un attributo del software usando un criterio di misurazione. Può aiutare a riconoscere dove l'evoluzione sta aumentando complessità e dipendenze, ma il suo valore va interpretato insieme a requisiti, responsabilità e comportamento del sistema.

11.2 Manutenzione ed evoluzione

11.2.1 Tipi e costo delle modifiche

Le modifiche successive alla consegna sono classificate in quattro categorie:

Correttive. Rimuovono errori o *bug*, cioè comportamenti non conformi a quanto previsto.

Adattative. Adeguano il sistema a un nuovo ambiente, per esempio a una piattaforma, a un sistema operativo o a hardware cambiato.

Perfettive. Migliorano il prodotto o aggiungono funzionalità.

Preventive. Modificano internamente il sistema per prevenire problemi futuri; il refactoring ne è un esempio.

La manutenzione può assorbire una parte molto rilevante del costo complessivo: il 67–80% è un ordine di grandezza orientativo. Una ripartizione statistica esemplificativa assegna il 17% agli interventi correttivi, il 18% agli adattativi, il 60% ai perfettivi e il 5% ai preventivi.

Queste quote descrivono una possibile ripartizione, non prevedono la distribuzione in ogni progetto.

L'importanza delle estensioni spiega la scelta di preparare il software al cambiamento già durante il design: la parametrizzazione rende variabili valori o scelte, mentre l'incapsulamento nasconde decisioni interne dietro contratti stabili.

11.2.2 Costo e modelli di manutenzione

La partecipazione del team originario può ridurre il costo della manutenzione grazie alla conoscenza del sistema.

Se però chi sviluppa non ha responsabilità contrattuali sulle modifiche successive, può non avere incentivi sufficienti a preparare il software ai cambiamenti. Qualunque sia il team incaricato, interventi non controllati possono peggiorare progressivamente la struttura.

Si distinguono tre modalità di intervento:

Quick-fix. Applica cambiamenti direttamente al codice. È il modello più immediato, ma non impone un riesame della struttura.

Miglioramento iterativo. Esegue le modifiche dopo un'analisi del sistema esistente, controllandone la complessità e conservandone il design.

Riuso. Stabilisce i requisiti del nuovo sistema e riusa quanto più possibile del sistema esistente.

La distinzione chiarisce perché correggere un problema non equivale sempre a mantenere bene il software: la modifica deve anche mantenere il software comprensibile e consentire ulteriori cambiamenti.

11.2.3 Ristrutturazione e ricostruzione

Il **refactoring**, o *restructuring*, è un cambiamento che migliora la struttura interna senza alterare il comportamento del codice. Non aggiunge quindi una funzionalità e non è, da solo, la correzione di un difetto osservabile.

Il **reverse engineering** analizza un sistema per ricavare informazioni sul suo comportamento o sulla sua struttura; il **re-engineering** altera invece il sistema per ricostruirlo in una forma diversa. Queste definizioni separano rispettivamente miglioramento interno, conoscenza dell'esistente e ricostruzione.

11.3 Dinamiche e leggi di Lehman

Le *dinamiche di evoluzione* sono i processi di cambiamento di un sistema. Le leggi di Lehman derivano da studi empirici condotti da Lehman e Belady dal 1968, e poi confermati da studi successivi, su grandi sistemi realizzati in grandi aziende.

Non sono dunque regole dimostrate per qualsiasi programma.

Cambiamento continuo (1974). Un sistema deve essere adattato continuamente; altrimenti diventa progressivamente meno soddisfacente.

Aumento della complessità (1974). Durante l'evoluzione la struttura tende a diventare più complessa, salvo che si svolga lavoro esplicito per preservarla o semplificarla.

Auto-regolazione (1974). Attributi quali dimensione, intervallo fra release e numero di errori rilevati in ciascuna release sono approssimativamente invarianti.

Stabilità organizzativa (1978). Nel ciclo di vita di un sistema il tasso di sviluppo è circa costante e indipendente dalle risorse impiegate.

Conservazione di familiarità (1978). In media l'incremento di crescita del sistema tende a restare costante oppure a diminuire.

Continua crescita (1978). Per mantenere la soddisfazione dell'utente occorre incrementare continuamente il contenuto funzionale del sistema.

Diminuzione della qualità (1994). La qualità diminuisce se non è gestita e adattata rigorosamente durante i cambiamenti.

Precisazione

Il campo di applicazione riguarda soprattutto sistemi grandi prodotti da grandi organizzazioni. L'estensione delle stesse regolarità a piccoli prodotti, piccole organizzazioni o sistemi con molti COTS non è automaticamente giustificata: cambiano dimensione, controllo sul codice e modalità di evoluzione.

11.4 Metriche del software

11.4.1 Scopo e limiti della misurazione

Una metrica definisce un insieme di misure per un sistema software. Gli obiettivi sono monitorare il prodotto durante la costruzione, individuare il livello raggiunto da ciascuna metrica e intervenire quando il livello non è soddisfacente.

Gli **attributi interni**, quali dimensione e struttura del codice, sono misurabili direttamente; gli **attributi esterni**, quali qualità percepita o manutenibilità, sono ricavati indirettamente.

Perciò un valore non è una prova automatica di qualità o di difetto, ma un'indicazione da interpretare nel contesto del progetto.

11.4.2 Dimensione e commenti

LOC (*Lines Of Code*) misura la dimensione in linee di codice. NCNB (*Non Comment Non Blank*) esclude invece commenti e righe vuote.

Il criterio usato per contare le linee deve essere dichiarato: nel caso di `CalcolaImporti` considerato qui, LOC conta le linee che terminano con `;`. Il confronto fra versioni e progetti è significativo soltanto se mantiene la medesima convenzione.

Percentuale di commenti e confronto tra versioni

La **Comment Percentage** esprime la percentuale di commenti rispetto alle LOC. Un riferimento del 30% può orientare una valutazione, ma non è una soglia universale: conta soprattutto che i commenti chiariscano intenzioni e decisioni.

Si considerino `CalcolaImporti` 0.1, con un metodo e 18 LOC, e la versione 0.2, con un metodo e 23 LOC. `Pagamenti` 1.1 ha invece otto metodi e 27 LOC: la media è $27/8 = 3{,}375$, circa 3,4 LOC per metodo. Queste misure indicano quantità; da sole non descrivono quanto sia difficile il comportamento del codice.

Un confronto quantitativo esemplificativo considera anche due basi di codice:

Sistema	LOC	Classi	Metodi	Attributi	LOC/metodo
JUnit	circa 22000	231	1200	265	circa 18
JHotDraw	circa 28000	600	4814	1151	circa 6

Questi dati descrivono specifiche basi di codice secondo determinate convenzioni di conteggio, non le versioni correnti delle librerie. La diversa media non dimostra da sola che uno dei due progetti sia migliore: occorre considerare responsabilità e struttura.

Precisazione

Un metodo breve può avere molte dipendenze o una condizione difficile da testare; un metodo più lungo può contenere una sequenza semplice. Per scegliere il refactoring si combinano quindi dimensione, coesione, accoppiamento e responsabilità.

Preferire metodi sotto le 30 righe è un criterio orientativo per riesaminare la leggibilità, non un limite di correttezza. Anche un valore medio di circa 18 LOC per metodo, associato a JUnit come esempio comparativo, richiede di precisare versione e convenzione di conteggio prima di usarlo per un confronto.

Un numero di righe va sempre letto insieme alle responsabilità del metodo.

La complessità ciclomatica affianca le misure di dimensione: considera i percorsi indipendenti del flusso di controllo. Il collegamento fra complessità ciclomatica e test permette di distinguere una classe corta ma ricca di decisioni da una classe più lunga con flusso semplice. Le metriche delle classi completano questa analisi considerando le dipendenze e l'organizzazione a oggetti.

11.5 Metriche CK per sistemi a oggetti

La suite di Chidamber e Kemerer, o *metriche CK*, misura aspetti delle classi di un sistema orientato agli oggetti. Le metriche non sono giudizi isolati: rendono visibili, da prospettive diverse, numero e complessità dei metodi, profondità e ampiezza dell'ereditarietà, dipendenze, risposta ai messaggi e coesione.

11.5.1 Metodi ed ereditarietà

WMC (Weighted Methods per Class). È la somma dei pesi di complessità dei metodi: $WMC = \sum_{j=1}^{m} c_j$, con m metodi e peso c_j del metodo j. Se ogni peso è unitario, WMC coincide con il numero dei metodi; con un peso comune c vale invece cm. Un WMC alto può segnalare maggiore lavoro di manutenzione e maggiore specificità della classe, con minori possibilità di riuso.

DIT (Depth of Inheritance Tree). È il massimo numero di livelli dalla classe alla radice della gerarchia; la radice ha livello zero. Un DIT alto può rendere più difficile comprendere

il comportamento, perché aumentano i metodi ereditati da considerare; può però indicare maggiore riuso.

NOC (Number of Children of a Class). È il numero di sottoclassi dirette di una classe. Un NOC maggiore può indicare più riuso e rende più ampia l'influenza della classe sul sistema: modificare una classe con molte discendenti può incidere su più comportamenti.

11.5.2 Accoppiamento, risposta e coesione

CBO (Coupling Between Object Classes). È il numero di classi con cui una classe interagisce. Un CBO alto esprime più dipendenze da altre classi e può rendere più difficile riusare, comprendere, modificare e correggere la classe.

RFC (Response for a Class). È il numero di metodi potenzialmente eseguibili in risposta a un messaggio: comprende i metodi della classe e i metodi invocati da ciascuno di essi. Un RFC alto segnala un insieme più ampio di comportamenti da comprendere e verificare.

LCOM (Lack of Cohesion of Methods). Il nome indica la mancanza di coesione dei metodi. Per interpretare un valore occorre precisare la formula adottata. Si adotta qui la definizione normalizzata:

$$LCOM = 1 - \frac{\sum_{i=1}^{a} mA_i}{a \cdot m}. \tag{11.1}$$

Qui a è il numero di campi della classe, m il numero di metodi e mA_i il numero di metodi che usano il campo A_i, dove i varia da 1 ad a. Il numeratore conta le coppie campo–metodo collegate; il denominatore $a \cdot m$ è il numero massimo di tali coppie. La formula richiede $a > 0$ e $m > 0$. Indicando con r il rapporto sottratto, r misura la densità d'uso dei campi e $LCOM = 1 - r$ ne è il complemento.

Con tre campi e tre metodi si ottengono quattro casi istruttivi:

Uso dei campi	Rapporto r	$LCOM = 1 - r$
Ogni metodo usa tutti e tre i campi	$9/9 = 1$	0
Ogni metodo usa due campi	$6/9 = 2/3$	$1/3$
Ogni metodo usa un campo	$3/9 = 1/3$	$2/3$
Un solo metodo usa tutti i campi; gli altri nessuno	$3/9 = 1/3$	$2/3$

Rapporto di uso e mancanza di coesione

I valori 1, 2/3, 1/3, 1/3 appartengono al rapporto d'uso, mentre l'indice di mancanza di coesione assume rispettivamente 0, 1/3, 2/3, 2/3. Non si devono scambiare le due quantità: per questa definizione un LCOM basso indica maggiore condivisione dei campi. Gli ultimi due casi mostrano anche un limite: strutture diverse possono produrre lo stesso valore. La metrica orienta quindi l'esame delle responsabilità, senza sostituirlo.

11.6 Difetti, metriche e interventi

Un **difetto** è riconoscibile confrontando il comportamento del programma con una *specifica*, cioè con la descrizione di ciò che esso deve fare: un comportamento anomalo segnala che il programma non soddisfa tale riferimento.

Si distinguono difetti di specifica, quando le specifiche sono ambigue o imprecise; difetti di progettazione, quando le componenti sono progettate non correttamente; e difetti di codice, derivanti dall'implementazione.

Metriche e test hanno ruoli complementari. LOC, *cc* e CK osservano proprietà interne e aiutano a localizzare classi o metodi che meritano analisi; i test esercitano il comportamento rispetto ai risultati attesi.

La correzione di un difetto richiede di: localizzare l'errore, correggere prima il modello e poi il codice, infine rieseguire il test.

Se non vi è un difetto di comportamento ma la struttura ostacola le modifiche, il refactoring è l'intervento appropriato, protetto dai test di regressione.

11.7 Esempio integrato

Si consideri una classe `Pagamenti` con otto metodi e 27 LOC, secondo il conteggio appena considerato. Il suo valore medio di 3,4 LOC per metodo non basta a stabilire se la classe sia facile da mantenere. Si procede allora con misure complementari: si calcola WMC sommando le complessità degli otto metodi; si osserva CBO per sapere quante altre classi coinvolge; si determina RFC contando i metodi attivati da un messaggio; si esamina la relazione fra campi e metodi, esplicitando la convenzione di coesione prima di attribuirle un valore LCOM.

Se uno dei metodi contiene una condizione, il suo grafo di controllo può avere due percorsi indipendenti e $cc = 2$; si preparano pertanto un test per ciascun esito.

Se il test rivela un risultato diverso dalla specifica, si tratta di un candidato difetto: si corregge modello e codice e si ripete il test.

Se invece i test restano corretti ma CBO o WMC richiamano l'attenzione su una struttura molto dipendente o complessa, il team può analizzare l'opportunità di una ristrutturazione. I valori alti o bassi vanno interpretati nel contesto: non esistono soglie universali ricavabili dal solo nome di una metrica.